

Aula 14 — Do colaborador concreto ao contrato

Como aceitar outras entregas sem fazer Pedido conhecer cada classe concreta?

Pedido já funciona com a entrega normal

```
private EntregaNormal entrega;

public void fechar(EntregaNormal entrega) {
    if (!fechado && entrega != null) {
        this.entrega = entrega;
        fechado = true;
    }
}
```

Nasce aberto e sem entrega; guarda a escolha no primeiro fechamento válido.

ATIVIDADE

UMA SEGUNDA CLASSE PARA O MESMO TRABALHO

A expressa cabe neste fechamento?

EntregaExpressa também oferece `calcularCusto(Pedido)` .

```
Pedido pedido = new Pedido();  
EntregaExpressa expressa = new EntregaExpressa();  
  
pedido.fechar(expressa);
```

Compila? Qual declaração determina se esse argumento é aceito?

O IMPEDIMENTO ESTÁ NO TIPO DECLARADO

Saber calcular ainda não basta para entrar

```
public void fechar(EntregaNormal entrega)
```

O parâmetro exige `EntregaNormal` ; o argumento é `EntregaExpressa` .

As duas sabem calcular custo, mas a chamada **não compila**.

Que capacidade Pedido realmente precisa solicitar?

COMPARE AS ALTERNATIVAS CONHECIDAS

A operação é comum; a regra de custo varia

Todas oferecem:

```
public double calcularCusto(Pedido pedido)
```

Classe	Resultado neste exemplo
EntregaNormal	10.0
EntregaExpressa	25.0
RetiradaLocal	0.0

ATIVIDADE

DESCREVA O PAPEL ANTES DA CONSTRUÇÃO DE JAVA

Complete sem citar uma classe

“Para calcular o custo, Pedido precisa de algo capaz de...”

Que operação esse colaborador deve oferecer? Que resultado deve devolver?

CONCEITO-CHAVE

A NECESSIDADE AGORA TEM UMA DESCRIÇÃO

Conceito-chave — contrato

Um contrato descreve o que um objeto precisa oferecer para ocupar um papel na colaboração.

Aqui: **calcular o custo da entrega de um pedido.**

Classes diferentes podem cumprir esse papel com regras diferentes.

DAR UM NOME DE TIPO AO PAPEL

Entrega.java declara o contrato

```
public interface Entrega {  
    double calcularCusto(Pedido pedido);  
}
```

Entrega nomeia a capacidade comum. Ainda não calcula custo algum.

Java em foco — interface

```
double calcularCusto(Pedido pedido);
```

- Resultado: `double` ; parâmetro: `Pedido` .
- A assinatura termina em `;` , sem corpo neste contrato.
- Essa operação é pública mesmo sem escrever `public` .

As classes concretas precisam oferecer uma implementação pública compatível.

AS CLASSES ASSUMEM A PROMESSA

A normal conserva sua regra

EntregaNormal.java:

```
public class EntregaNormal implements Entrega {  
    public double calcularCusto(Pedido pedido) {  
        return 10.0;  
    }  
}
```

`implements Entrega` declara que a classe cumpre o contrato.

O MESMO CONTRATO TAMBÉM ADMITE OUTRA REGRA

A expressa continua calculando 25.0

EntregaExpressa.java:

```
public class EntregaExpressa implements Entrega {  
    public double calcularCusto(Pedido pedido) {  
        return 25.0;  
    }  
}
```

A classe mudou sua declaração, mantendo a implementação do cálculo.

ATIVIDADE

A RETIRADA TAMBÉM PRECISA CUMPRIR A PROMESSA

Esta classe concreta compila?

RetiradaLocal.java, numa tentativa incompleta:

```
public class RetiradaLocal implements Entrega {  
}
```

O que falta? Ao completar, precisa devolver o mesmo valor da normal?

PROMETER EXIGE OFERECER A OPERAÇÃO

A retirada cumpre o contrato com custo zero

```
public class RetiradaLocal implements Entrega {  
    public double calcularCusto(Pedido pedido) {  
        return 0.0;  
    }  
}
```

O contrato exige a operação, sem fixar um valor único.

AS CLASSES JÁ CUMPREM ENTREGA

Pedido ainda precisa aceitar esse papel

Antes:

```
private EntregaNormal entrega;  
public void fechar(EntregaNormal entrega)
```

Depois:

```
private Entrega entrega;  
public void fechar(Entrega entrega)
```

MUDA O TIPO; A REGRA DE FECHAMENTO PERMANECE

Pedido continua guardando a primeira escolha válida

```
private Entrega entrega;

public void fechar(Entrega entrega) {
    if (!fechado && entrega != null) {
        this.entrega = entrega;
        fechado = true;
    }
}
```

O construtor continua sem entrega. A colaboração começa em `fechar(...)`.

Pedido continua delegando o cálculo

```
public double calcularCustoEntrega() {  
    return entrega.calcularCusto(this);  
}
```

Consulta **depois de um fechamento válido**.

`this` fornece o próprio pedido ao colaborador. O total continua somando somente itens.

ATIVIDADE

RETOME A TENTATIVA QUE ABRIU A AULA

A mesma chamada agora funciona?

```
Pedido pedido = new Pedido();  
EntregaExpressa expressa = new EntregaExpressa();  
  
pedido.fechar(expressa);  
System.out.println(pedido.calcularCustoEntrega());
```

Agora compila? Qual valor aparece? Quem calcula esse valor?

O CONFLITO INICIAL FOI RESOLVIDO

A expressa entrou sem levar sua fórmula a Pedido

25.0

- EntregaExpressa cumpre Entrega .
- fechar aceita uma referência do tipo Entrega .
- O custo continua sendo calculado pelo colaborador.

Aceitar a expressa resolveu o encaixe. E o que já funcionava?

A AULA 13 PREPAROU A VERIFICAÇÃO DESSA MUDANÇA

Execute novamente a suíte conhecida

Teste	Expectativa preservada
<code>calculaTotalDeUmItem()</code>	<code>300.0</code> para duas unidades a <code>150.0</code>
<code>pedidoFechadoNaoAceitaNovosItens()</code>	inclusão recusada: total <code>150.0</code>
<code>entregaNormalCustaDezReais()</code>	custo normal <code>10.0</code>

As entradas e expectativas continuam iguais. Os três testes devem passar.

OBSERVE UMA DAS EXPECTATIVAS PRESERVADAS

O teste da normal continua igual

```
@Test
void entregaNormalCustaDezReais() {
    Pedido pedido = new Pedido();
    pedido.fechar(new EntregaNormal());

    assertEquals(10.0, pedido.calcularCustoEntrega());
}
```

O tipo do campo mudou. O comportamento público esperado permaneceu.

OUTRA CLASSE AGORA CABE; A ESCOLHA AINDA É CONSERVADA

Uma nova verificação protege o segundo fechamento

```
@Test
void segundoFechamentoNaoTrocaEntrega() {
    Pedido pedido = new Pedido();
    pedido.fechar(new EntregaNormal());
    pedido.fechar(new EntregaExpressa());

    assertEquals(10.0, pedido.calcularCustoEntrega());
}
```

A expressa é um argumento aceito. A guarda ainda impede trocar a escolha.

ATIVIDADE

LEIA O ALCANCE DAS VERIFICAÇÕES

Testes verdes demonstram o quê?

Os testes de custo normal e segundo fechamento passaram.

1. Quais comportamentos esses dois cenários verificaram?
2. Eles também verificaram o custo expresso e toda edição de itens?

A MUDANÇA FOI VERIFICADA NOS CENÁRIOS ESCOLHIDOS

Cada teste protege uma expectativa

Os dois testes verificam custo normal e conservação da escolha.

A suíte retomada também verifica total simples e uma inclusão recusada.

Outras operações exigem seus próprios cenários.

Agora chegou uma entrega que Pedido ainda não conhece.

PONHA O BENEFÍCIO DO CONTRATO À PROVA

Entrega agendada: custo fixo de R\$ 15

EntregaAgendada.java:

```
public class EntregaAgendada implements Entrega {  
    public double calcularCusto(Pedido pedido) {  
        return 15.0;  
    }  
}
```

Essa classe cumpre o papel que Pedido já solicita.

ATIVIDADE

USE A IMPLEMENTAÇÃO NOVA NO MESMO PONTO

Qual é o impacto em Pedido?

```
Pedido pedido = new Pedido();  
pedido.fechar(new EntregaAgendada());  
  
System.out.println(pedido.calcularCustoEntrega());
```

Que valor aparece? Precisamos mudar campo, fechamento ou consulta em Pedido?

O NOVO OBJETO JÁ CABE NO CONTRATO

A saída é 15.0; Pedido permaneceu igual

Parte do programa	Impacto da entrega agendada
<code>EntregaAgendada.java</code>	nova classe que cumpre <code>Entrega</code>
<code>Main</code> ou outro cliente	escolhe, cria e fornece o objeto
<code>Entrega.java</code>	conserva a operação do contrato
<code>Pedido.java</code>	conserva campo, fechamento e consulta

A escolha concreta continua existindo fora de Pedido.

CONCEITO-CHAVE

NOMEIE O GANHO QUE ACABAMOS DE OBSERVAR

Conceito-chave — depender de uma capacidade

Pedido depende do contrato `Entrega`, que expressa a capacidade estável de calcular o custo.

As classes concretas podem variar sem que Pedido precise nomear cada uma.

A consulta é a mesma; os resultados variam

Em cada pedido fechado:

```
pedido.calcularCustoEntrega()
```

Colaborador	Resultado
EntregaNormal	10.0
EntregaExpressa	25.0
EntregaAgendada	15.0

Como um campo do tipo Entrega pode alcançar esses objetos diferentes?

ATIVIDADE

RECUPERE VARIÁVEL, REFERÊNCIA E OBJETO

O que foi criado? Qual é o tipo declarado?

```
Entrega entrega = new EntregaExpressa();
```

1. Qual é a classe do objeto criado por `new` ?
2. Qual é o tipo declarado da variável?
3. Declarar `Entrega` mudou a classe do objeto?

O objeto continua concreto

```
Entrega entrega = new EntregaExpressa();
```



- Tipo declarado da variável: `Entrega` .
- Classe do objeto: `EntregaExpressa` .
- A seta significa **aponta para**; a variável guarda uma referência.

O CONTRATO PERMITE EXPRESSAR A COLABORAÇÃO

Podemos fornecer essa referência ao pedido

```
Entrega entrega = new EntregaExpressa();  
Pedido pedido = new Pedido();  
pedido.fechar(entrega);
```

O campo guarda uma referência para o objeto concreto escolhido.

O tipo `Entrega` permite solicitar `calcularCusto(Pedido)` por essa referência.

A expressa pode oferecer mais operações

```
public class EntregaExpressa implements Entrega {  
    public double calcularCusto(Pedido pedido) {  
        return 25.0;  
    }  
  
    public void priorizarNaFila() {  
        // comportamento específico desta classe  
    }  
}
```

`priorizarNaFila()` não foi declarado em `Entrega`.

ATIVIDADE

PARA APROFUNDAR — O TIPO LIMITA A CHAMADA DISPONÍVEL

O objeto tem o método. A chamada compila?

```
Entrega entrega = new EntregaExpressa();  
entrega.priorizarNaFila();
```

E se a variável fosse declarada como `EntregaExpressa` ?

PARA APROFUNDAR — O OBJETO PRESERVA SUAS OPERAÇÕES

Muda o que podemos solicitar pela referência

```
Entrega entrega = new EntregaExpressa();  
entrega.priorizarNaFila(); // não compila
```

```
EntregaExpressa expressa = new EntregaExpressa();  
expressa.priorizarNaFila(); // compila
```

O primeiro tipo não declara essa operação; o segundo a oferece.

ATIVIDADE

PARA APROFUNDAR — TIPO ACEITO E REGRA DE ESTADO

A interface permite trocar a escolha depois de fechar?

```
Pedido pedido = new Pedido();  
pedido.fechar(new EntregaNormal());  
pedido.fechar(new EntregaExpressa());  
  
System.out.println(pedido.calcularCustoEntrega());
```

Preveja a saída. Que condição explica o resultado?

PARA APROFUNDAR — O PRIMEIRO FECHAMENTO FOI CONSERVADO

Aceitar o argumento não autoriza substituir a entrega

```
if (!fechado && entrega != null) {  
    this.entrega = entrega;  
    fechado = true;  
}
```

Na segunda chamada, `!fechado` é falso. O custo permanece **10.0**.

O contrato ampliou os tipos aceitos; a regra de estado permaneceu.

ATIVIDADE

PARA APROFUNDAR — O CONTRATO RESPONDE A UMA NECESSIDADE

Precisamos de uma interface para cada classe?

Criaríamos `ProdutoInterface` ou `ItemPedidoInterface` apenas porque aprendemos `interface` ?

Que necessidade concreta procuraríamos antes dessa decisão?

Ter uma classe não exige uma interface correspondente

No caso de Entrega, já havia:

- um trabalho estável que Pedido precisava solicitar;
- classes diferentes para realizar esse trabalho;
- um impedimento para usá-las no mesmo ponto.

Essa necessidade ainda não foi mostrada para Produto ou ItemPedido.

SÍNTESE

DO IMPEDIMENTO À COLABORAÇÃO POR CONTRATO

O que a evolução conseguiu preservar e ampliar?

- Entrega nomeia o papel; implements declara quem o cumpre.
- Pedido aceita esse papel e mantém a regra de fechamento.
- Os testes verificam os comportamentos escolhidos.
- A entrega agendada entra sem modificar Pedido.
- A referência usa o contrato; o objeto continua concreto.

TRANSFIRA A DECISÃO PARA O PROJETO 2

No Laboratório 14, o papel será enviar mensagens

Nas aulas	No laboratório
Pedido solicita custo	Lembrete solicita envio
contrato Entrega	contrato Notificador
entrega escolhida no fechamento	canal inicial recebido na criação

Mudar o tipo de colaborador não decide quando o modelo deve recebê-lo.

A PERGUNTA QUE OS RESULTADOS DEIXARAM

Como Java escolhe qual cálculo executar?

Em Pedido, a chamada continua:

```
return entrega.calcularCusto(this);
```

O contrato é o mesmo; os objetos e os resultados podem ser diferentes.

Como Java sabe qual implementação executar?